

Unified memory management

Contents

- Unified memory
- Managed memory
- Memory allocation approaches in unified memory
- Using unified memory
- Performance optimizations for unified memory

This document covers unified memory management in HIP, which encompasses several approaches that provide a single address space accessible from both CPU and GPU. **Unified memory** refers to the overall architectural concept of this shared address space, while **managed memory** is one specific implementation that provides automatic page migration between devices. Other unified memory allocators like `hipMalloc()` and `hipHostMalloc()` provide different access patterns within the same unified address space concept.

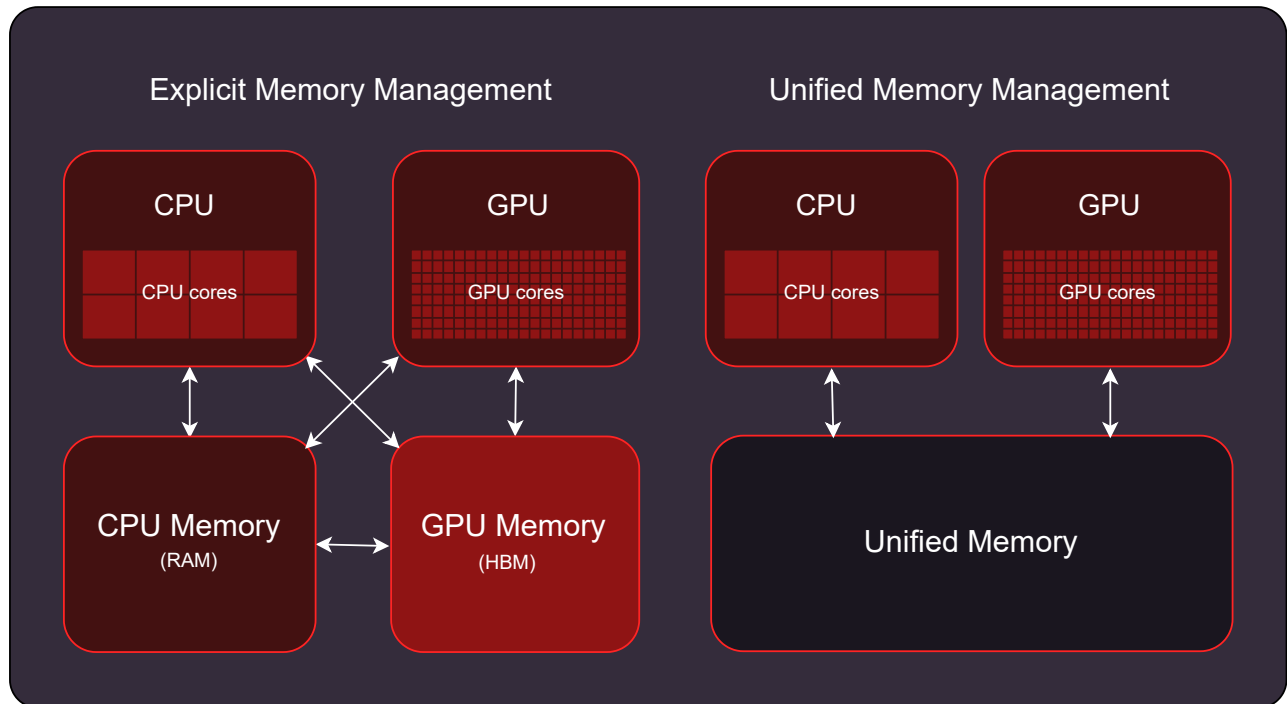
In conventional architectures CPUs and attached devices have their own memory space and dedicated physical memory backing it up, e.g. normal RAM for CPUs and VRAM on GPUs. This way each device can have physical memory optimized for its use case. GPUs usually have specialized memory whose bandwidth is a magnitude higher than the RAM attached to CPUs.

While providing exceptional performance, this setup typically requires explicit memory management, as memory needs to be allocated, copied and freed on the used devices and on the host. Additionally, this makes using more than the physically available memory on the devices complicated.

Modern GPUs circumvent the problem of having to explicitly manage the memory, while still keeping the benefits of the dedicated physical memories, by supporting the concept of unified memory. This enables the CPU and the GPUs in the system to a  [latest](#) 
GPUs' memory without explicit memory management.

Unified memory

Unified Memory is a single memory address space accessible from any processor within a system. This setup simplifies memory management and enables applications to allocate data that can be read or written on both CPUs and GPUs without explicitly copying it to the specific CPU or GPU. The Unified memory model is shown in the following figure.



Unified memory enables the access to memory located on other devices via several methods, depending on whether hardware support is available or has to be managed by the driver. CPUs can access memory allocated via `hipMalloc()`, providing bidirectional memory accessibility within the unified address space.

Managed memory

Managed Memory is an extension of the unified memory architecture in which HIP monitors memory access and intelligently migrates data between device and system memories, thereby improving performance and resource efficiency.

When a kernel on the device tries to access a managed memory address that is not in its local device memory, a page-fault is triggered. The GPU then in turn requests the page from the host or other device on which the memory is located. The page is unmapped from the source, sent to the device and mapped to the device's memory. The requested memory is then available locally to the processes running on the device, which improves performance as local memory access outperforms remote memory access.

Managed memory also expands the memory capacity available to a GPU kernel. When migrating memory into the device on page-fault, if the device's memory is already at capacity, a page is unmapped from the device's memory first and sent and mapped to host memory. This enables more memory to be allocated and used for a GPU than the GPU itself has physically available. This level of support can be very beneficial, for example, for sparse accesses to an array that is not often used on the device.

System requirements for managed memory

Some AMD GPUs do not support page-faults, and thus do not support on-demand page-fault driven migration. On these architectures, if the programmer prefers all GPU memory accesses to be local, all pages have to be migrated before the kernel is dispatched, as the driver cannot know beforehand which parts of a dataset are going to be accessed. This can lead to significant delays on the first run of a kernel, and, in the example of a sparsely accessed array, can also lead to copying more memory than is actually accessed by the kernel.

Note that on systems which do not support page-faults, managed memory APIs are still accessible to the programmer, but managed memory operates in a degraded fashion due to the lack of demand-driven migration. Furthermore, on these systems it is still possible to use unified memory allocators that do not provide managed memory features; see [Memory allocation approaches in unified memory](#) for more details.

Managed memory is supported on Linux by all modern AMD GPUs from the Vega series onward, as shown in the following table. Managed memory can be explicitly allocated using `hipMallocManaged()` or marking variables with the `__managed__` attribute. For the latest GPUs, with a Linux kernel that supports [Heterogeneous Memory Management \(HMM\)](#), the normal system allocators (e.g., `new`, `malloc()`) can be used.

Note

To ensure the proper functioning of managed memory on supported GPUs, it is **essential** to set the environment variable `HSA_XNACK=1` and use a GPU kernel mode driver that supports [HMM](#). Without this configuration, access-driven memory migration will be disabled, and the behavior will be similar to that of systems without HMM support.

Managed Memory Support by GPU Architecture

Architecture	<code>hipMallocManaged()</code> , <code>__managed__</code>	<code>new</code> , <code>malloc()</code> , <code>allocate()</code>
CDNA4	✓	✓ ¹
CDNA3	✓	✓ ¹
CDNA2	✓	✓ ¹
CDNA1	✓	✗
RDNA1	✓	✗
GCN5	✓	✗

✓: **Supported**

✗: **Unsupported**

¹ Works only with `HSA_XNACK=1` and kernels with HMM support. First GPU access causes recoverable page-fault.

Memory allocation approaches in unified memory

While managed memory provides automatic migration, unified memory encompasses several allocation methods, each with different access patterns and migration behaviors. The following section covers all available unified memory allocation approaches, including but not limited to managed memory APIs.

Support for the different unified memory allocators depends on the GPU architecture and on the system. For more information, see [System requirements for managed memory](#) and [Checking unified memory support](#).

- **HIP allocated managed memory and variables**

`hipMallocManaged()` is a dynamic memory allocator available on all GPUs with unified memory support. For more details, visit [Managed memory](#).

The `__managed__` declaration specifier, which serves as its counterpart, can be utilized for static allocation.

 [latest](#) ▼

- **System allocated unified memory**

Starting with CDNA2, the `new`, `malloc()`, and `allocate()` (Fortran) system allocators allow you to reserve unified memory. The system allocator is more versatile and offers an easy transition for code written for CPUs to HIP code as the same system allocation API is used. Memory allocated by these allocators can be registered to be accessible on device using `hipHostRegister()`.

- **HIP allocated non-managed memory**

`hipMalloc()` and `hipHostMalloc()` are dynamic memory allocators available on all GPUs with unified memory support. Memory allocated by these allocators is not migrated between device and host memory.

The table below illustrates the expected behavior of managed and unified memory functions on ROCm and CUDA, both with and without HMM support.

ROCm allocation behaviour		CUDA allocation behaviour		
Comparison of expected behavior of managed and unified memory functions in ROCm				
call	Allocation origin without HMM or HSA_XNACK=0	Access outside the origin without HMM or HSA_XNACK=0	Allocation origin with HMM and HSA_XNACK=1	Access outside the origin with HMM and HSA_XNACK=1
new , malloc() , allocate()	host	not accessible on device	first touch	page-fault migration
hipMalloc()	device	zero copy [zc]	device	zero copy [zc]
hipMallocManaged() , __managed__	pinned host	zero copy [zc]	first touch	page-fault migration
hipHostRegister()	pinned host	zero copy [zc]	pinned host	zero copy [zc]
hipHostMalloc()	pinned host	zero copy [zc]	pinned host	zero copy [zc]

[\[zc\]\(1,2,3,4,5,6,7,8,9\)](#) Zero copy is a feature, where the memory is pinned to the device or the host, and won't be transferred when accessed by another device or the host.

Instead only the requested memory is transferred, without making an explicit copy, like a normal memory access, hence the term “zero copy”.

Checking unified memory support

The following device attributes can offer information about which [Memory allocation approaches in unified memory](#) are supported. The attribute value is 1 if the functionality is supported, and 0 if it is not supported.

Device attributes for unified memory management

Attribute	Description
<code>hipDeviceAttributeManagedMemory</code>	Device supports allocating managed memory on this system
<code>hipDeviceAttributePageableMemoryAccess</code>	Device supports coherently accessing pageable memory without calling <code>hipHostRegister()</code> on it.
<code>hipDeviceAttributeConcurrentManagedAccess</code>	Full unified memory support. Device can coherently access managed memory concurrently with the CPU

For details on how to get the attributes of a specific device see `hipDeviceGetAttribute()`.

Example for unified memory management

The following example shows how to use unified memory with `hipMallocManaged()` for dynamic allocation, the `__managed__` attribute for static allocation and the standard `new` allocation. For comparison, the explicit memory management example is presented in the last tab.

`hipMallocManaged()`

`__managed__`

`new`

Explicit Memory Management

```

#include <hip/hip_runtime.h>

#include <iostream>

#define HIP_CHECK(expression) \
{ \
    const hipError_t err = expression; \
    if(err != hipSuccess) \
    { \
        std::cerr << "HIP error: " \
        << hipGetErrorString(err) \
        << " at " << __LINE__ << "\n"; \
    } \
}

// Addition of two values.
__global__ void add(int *a, int *b, int *c)
{
    *c = *a + *b;
}

int main()
{
    int *a, *b, *c;

    // Allocate memory for a, b and c that is accessible to both device and host
    HIP_CHECK(hipMallocManaged(&a, sizeof(*a)));
    HIP_CHECK(hipMallocManaged(&b, sizeof(*b)));
    HIP_CHECK(hipMallocManaged(&c, sizeof(*c)));

    // Setup input values.
    *a = 1;
    *b = 2;

    // Launch add() kernel on GPU.
    add<<<1, 1>>>(a, b, c);

    // Wait for GPU to finish before accessing on host.
    HIP_CHECK(hipDeviceSynchronize());

    // Print the result.
    std::cout << *a << " + " << *b << " = " << *c << std::endl;

    // Cleanup allocated memory.
    HIP_CHECK(hipFree(a));
    HIP_CHECK(hipFree(b));
    HIP_CHECK(hipFree(c));

    return 0;
}

```

Choosing the right memory allocator

Selecting the appropriate memory allocator depends on your specific use case, performance requirements, and target hardware. The following table provides guidance on when to use each unified memory allocator in HIP.

Memory allocator comparison

Allocator	Best Use Case	Key Characteristics
hipMallocManaged() / <code>__managed__</code>	Applications that need automatic data migration between host and device, or require memory oversubscription	Automatic page-fault driven migration; accessible from both host and device; may have performance overhead from on-demand migration
<code>new</code> , <code>malloc()</code> , <code>allocate()</code> (CDNA2+ with HMM)	CPU-centric code being ported to HIP, or applications that want system allocator simplicity with GPU accessibility	Uses standard system allocators; first-touch placement policy; requires <code>HSA_XNACK=1</code> for GPU access; convenient for porting
hipMalloc()	Device-local data that is primarily accessed by GPU kernels with minimal host interaction	Device-local allocation; zero-copy access from host; no automatic migration; best performance for GPU-intensive workloads
hipHostMalloc()	Data that is primarily accessed by the host but needs to be accessible from the device	Pinned host memory; zero-copy access from device; no automatic migration; can improve transfer bandwidth for host-device copies

When choosing an allocator, consider the following factors:

- **Access pattern:** If data is accessed frequently by both host and device, managed memory or system allocators provide automatic migration. If data is primarily used by one processor, [hipMalloc\(\)](#) or [hipHostMalloc\(\)](#) may be more efficient.
- **Performance requirements:** For maximum performance with known access patterns, explicit memory management with [hipMalloc\(\)](#) and [hipMemcpy\(\)](#) the best performance. Managed memory trades some performance for simplicity.

 [latest](#) ▼

- **Memory oversubscription:** Only managed memory and system allocators (with HMM support) allow allocating more memory than physically available on the device.
- **Portability:** System allocators (`new`, `malloc()`) make porting CPU code easier, but require CDNA2+ with `HSA_XNACK=1` for GPU access.

Using unified memory

Unified memory can simplify the complexities of memory management in GPU computing, by not requiring explicit copies between the host and the devices. It can be particularly useful in use cases with sparse memory accesses from both the CPU and the GPU, as only the parts of the memory region that are actually accessed need to be transferred to the corresponding processor, not the whole memory region. This reduces the amount of memory sent over the PCIe bus or other interfaces.

In HIP, pinned memory allocations are coherent by default. Pinned memory is host memory mapped into the address space of all GPUs, meaning that the pointer can be used on both host and device. Additionally, using pinned memory instead of pageable memory on the host can improve bandwidth for transfers between the host and the GPUs.

While unified memory can provide numerous benefits, it's important to be aware of the potential performance overhead associated with unified memory. You must thoroughly test and profile your code to ensure it's the most suitable choice for your use case.

Performance optimizations for unified memory

There are several ways, in which the developer can guide the runtime to reduce copies between devices, in order to improve performance.

With `numactl --membind` bindings, developers can control where physical allocation occurs by restricting memory allocation to specific NUMA nodes. This approach can reduce or eliminate the need for explicit data prefetching since memory is allocated in the desired location from the start.

Data prefetching

Warning

Data prefetching is not always an optimization and can slow down execution, as the API takes time to execute. If the memory is already in the right place, prefetching will waste time. Users should profile their code to verify whether prefetching is beneficial for their specific use case.

Note

Cold Start Latency: Implicit page faulting on first access can cause significant latency spikes, especially in performance-critical code paths. To avoid this, use `hipMemPrefetchAsync()` during initialization to ensure data is resident on the target device before critical kernels execute. This proactive approach eliminates the “cold start” penalty associated with on-demand page migration.

When prefetching is beneficial, developers can consider setting different default locations for different devices and using prefetch between them, which can help eliminate IPC communication overhead when memory moves between devices.

Data prefetching is a technique used to improve the performance of your application by moving data to the desired device before it's actually needed. `hipCpuDeviceId` is a special constant to specify the CPU as target.

```

#include <hip/hip_runtime.h>

#include <iostream>

#define HIP_CHECK(expression) \
{ \
    const hipError_t err = expression; \
    if(err != hipSuccess) \
    { \
        std::cerr << "HIP error: " \
            << hipGetErrorString(err) \
            << " at " << __LINE__ << "\n"; \
    } \
}

// Addition of two values.
__global__ void add(int *a, int *b, int *c)
{
    *c = *a + *b;
}

int main()
{
    int *a, *b, *c;
    int deviceId;
    HIP_CHECK(hipGetDevice(&deviceId)); // Get the current device ID

    // Allocate memory for a, b and c that is accessible to both device and host co
    HIP_CHECK(hipMallocManaged(&a, sizeof(*a)));
    HIP_CHECK(hipMallocManaged(&b, sizeof(*b)));
    HIP_CHECK(hipMallocManaged(&c, sizeof(*c)));

    // Setup input values.
    *a = 1;
    *b = 2;

    // Prefetch the data to the GPU device.
    HIP_CHECK(hipMemPrefetchAsync(a, sizeof(*a), deviceId, 0));
    HIP_CHECK(hipMemPrefetchAsync(b, sizeof(*b), deviceId, 0));
    HIP_CHECK(hipMemPrefetchAsync(c, sizeof(*c), deviceId, 0));

    // Launch add() kernel on GPU.
    add<<<1, 1>>>(a, b, c);

    // Prefetch the result back to the CPU.
    HIP_CHECK(hipMemPrefetchAsync(c, sizeof(*c), hipCpuDeviceId, 0));

    // Wait for the prefetch operations to complete.
    HIP_CHECK(hipDeviceSynchronize());

    // Prints the result.
    std::cout << *a << " + " << *b << " = " << *c << std::endl;

    // Cleanup allocated memory.
    HIP_CHECK(hipFree(a));
    HIP_CHECK(hipFree(b));
    HIP_CHECK(hipFree(c));

```

```
    return 0;  
}
```

Memory advice

Unified memory runtime hints can be set with `hipMemAdvise()` to help improve the performance of your code if you know the memory usage pattern. There are several different types of hints as specified in the enum `hipMemoryAdvise`, for example, whether a certain device mostly reads the memory region, where it should ideally be located, and even whether that specific memory region is accessed by a specific device.

For the best performance, profile your application to optimize the utilization of HIP runtime hints.

The effectiveness of `hipMemAdvise()` comes from its ability to inform the runtime of the developer's intentions regarding memory usage. When the runtime has knowledge of the expected memory access patterns, it can make better decisions about data placement, leading to less transfers via the interconnect and thereby reduced latency and bandwidth requirements. However, the actual impact on performance can vary based on the specific use case and the system.

The following is the updated version of the example above with memory advice instead of prefetching.

```

#include <hip/hip_runtime.h>
#include <iostream>

#define HIP_CHECK(expression) \
{ \
    const hipError_t err = expression; \
    if(err != hipSuccess) \
    { \
        std::cerr << "HIP error: " \
        << hipGetErrorString(err) \
        << " at " << __LINE__ << "\n"; \
    } \
}

// Addition of two values.
__global__ void add(int *a, int *b, int *c)
{
    *c = *a + *b;
}

int main()
{
    int deviceId;
    HIP_CHECK(hipGetDevice(&deviceId));
    int *a, *b, *c;

    // Allocate memory for a, b, and c accessible to both device and host codes.
    HIP_CHECK(hipMallocManaged(&a, sizeof(*a)));
    HIP_CHECK(hipMallocManaged(&b, sizeof(*b)));
    HIP_CHECK(hipMallocManaged(&c, sizeof(*c)));

    // Set memory advice for a and b to be read, located on and accessed by the GPU
    HIP_CHECK(hipMemAdvise(a, sizeof(*a), hipMemAdviseSetPreferredLocation, deviceId));
    HIP_CHECK(hipMemAdvise(a, sizeof(*a), hipMemAdviseSetAccessedBy, deviceId));
    HIP_CHECK(hipMemAdvise(a, sizeof(*a), hipMemAdviseSetReadMostly, deviceId));

    HIP_CHECK(hipMemAdvise(b, sizeof(*b), hipMemAdviseSetPreferredLocation, deviceId));
    HIP_CHECK(hipMemAdvise(b, sizeof(*b), hipMemAdviseSetAccessedBy, deviceId));
    HIP_CHECK(hipMemAdvise(b, sizeof(*b), hipMemAdviseSetReadMostly, deviceId));

    // Set memory advice for c to be read, located on and accessed by the CPU.
    HIP_CHECK(hipMemAdvise(c, sizeof(*c), hipMemAdviseSetPreferredLocation, hipCpuDeviceId));
    HIP_CHECK(hipMemAdvise(c, sizeof(*c), hipMemAdviseSetAccessedBy, hipCpuDeviceId));
    HIP_CHECK(hipMemAdvise(c, sizeof(*c), hipMemAdviseSetReadMostly, hipCpuDeviceId));

    // Setup input values.
    *a = 1;
    *b = 2;

    // Launch add() kernel on GPU.
    add<<<1, 1>>>(a, b, c);

    // Wait for GPU to finish before accessing on host.
    HIP_CHECK(hipDeviceSynchronize());

    // Prints the result.
    std::cout << *a << " + " << *b << " = " << *c << std::endl;
}

```

 latest ▼

```
// Cleanup allocated memory.  
HIP_CHECK(hipFree(a));  
HIP_CHECK(hipFree(b));  
HIP_CHECK(hipFree(c));  
  
return 0;  
}
```

Best practices for memory advice

Choosing the right memory advice hint can significantly improve performance by reducing unnecessary data migrations. The following table provides guidance on when to use specific hints:

Memory advice best practices

Hint	Use Case
<code>hipMemAdviseSetReadMostly</code>	Data that is mostly read and only occasionally written to. This prevents unnecessary data migration for read-heavy workloads.
<code>hipMemAdviseSetPreferredLocation</code>	Set the preferred location for the data as a specific device. Use this to keep data on a particular device (host or device) to reduce migration overhead.
<code>hipMemAdviseSetAccessedBy</code>	Data that will be accessed by a specified device. This helps prevent page faults by proactively migrating data to the accessing device.
<code>hipMemAdviseSetCoarseGrain</code>	Data that only needs to be coherent at dispatch boundaries. The default fine-grained model allows coherent operations during kernel execution, while coarse-grained provides better performance for data that does not require runtime coherence.

Memory range attributes

`hipMemRangeGetAttribute()` allows you to query attributes of a given memory range. The attributes are given in `hipMemRangeAttribute`.

```

#include <hip/hip_runtime.h>

#include <cstdint>
#include <iostream>

#define HIP_CHECK(expression) \
{ \
    const hipError_t err = expression; \
    if(err != hipSuccess) \
    { \
        std::cerr << "HIP error: " \
        << hipGetErrorString(err) \
        << " at " << __LINE__ << "\n"; \
    } \
}

// Addition of two values.
__global__ void add(int *a, int *b, int *c)
{
    *c = *a + *b;
}

int main()
{
    int *a, *b, *c;
    unsigned int attributeValue;
    constexpr std::size_t attributeSize = sizeof(attributeValue);

    int deviceId;
    HIP_CHECK(hipGetDevice(&deviceId));

    // Allocate memory for a, b and c that is accessible to both device and host code.
    HIP_CHECK(hipMallocManaged(&a, sizeof(*a)));
    HIP_CHECK(hipMallocManaged(&b, sizeof(*b)));
    HIP_CHECK(hipMallocManaged(&c, sizeof(*c)));

    // Setup input values.
    *a = 1;
    *b = 2;

    HIP_CHECK(hipMemAdvise(a, sizeof(*a), hipMemAdviseSetReadMostly, deviceId));

    // Launch add() kernel on GPU.
    add<<<1, 1>>>(a, b, c);

    // Wait for GPU to finish before accessing on host.
    HIP_CHECK(hipDeviceSynchronize());

    // Query an attribute of the memory range.
    HIP_CHECK(hipMemRangeGetAttribute(&attributeValue,
                                      attributeSize,
                                      hipMemRangeAttributeReadMostly,
                                      a,
                                      sizeof(*a)));

    // Prints the result.
    std::cout << *a << " + " << *b << " = " << *c << std::endl;
    std::cout << "The array a is" << (attributeValue == 1 ? "" : " NOT") << " set to" << attributeValue << std::endl;
}

```

 latest ▼


```
// Cleanup allocated memory.  
HIP_CHECK(hipFree(a));  
HIP_CHECK(hipFree(b));  
HIP_CHECK(hipFree(c));  
  
return 0;  
}
```

Asynchronously attach memory to a stream

The `hipStreamAttachMemAsync()` function attaches memory to a stream, which can reduce the amount of memory transferred, when managed memory is used. When the memory is attached to a stream using this function, it only gets transferred between devices, when a kernel that is launched on this stream needs access to the memory.

[Terms and Conditions](#)

[ROCm Licenses and Disclaimers](#)

[Privacy](#)

[Trademarks](#)

[Supply Chain Transparency](#)

[Fair and Open Competition](#)

[UK Tax Strategy](#)

[Cookie Policy](#)

[Cookie Settings](#)

© 2026 Advanced Micro Devices, Inc